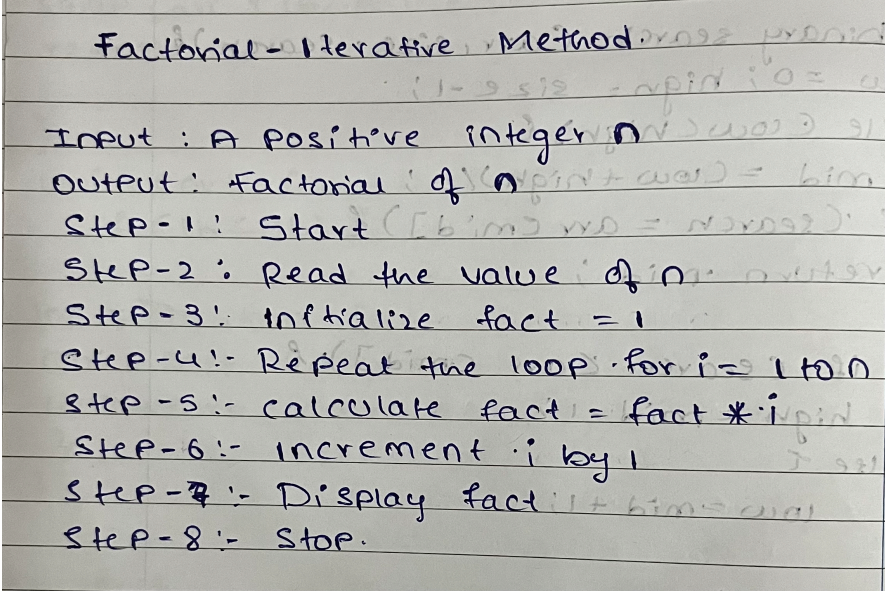


PRACTICAL - 4

AIM : Implementation and Time analysis of factorial program using iterative and recursive method

ALGORITHM : // Factorial Iterative method



Factorial - Iterative Method

Input : A positive integer n

Output : Factorial of n

Step-1 : Start

Step-2 : Read the value of n

Step-3 : Initialize $fact = 1$

Step-4 : Repeat the loop for $i = 1$ to n

Step-5 : calculate $fact = fact * i$

Step-6 : increment i by 1

Step-7 : Display $fact$

Step-8 : Stop

PROGRAM :

```
#include <iostream>
using namespace std;

long long factorial(int n) {x
    long long result = 1;

    for (int i = 1; i <= n; i++) {
        result = result * i;

        cout << "Step " << i << ": "
             << result << endl;
    }

    return result;
}

int main() {
    int n;
```

```
cout << "Enter a number: ";  
cin >> n;  
  
cout << "\n--- Iterative Process ---" << endl;  
  
long long result = factorial(n);  
  
cout << "\nFactorial of " << n << " = " << result << endl;  
  
return 0;  
}
```

OUTPUT :

```
SYNC STORAGE/UNIVERSITY STUFF/DAA_LAB  
> ./FactorialIterative  
Enter a number: 5  
  
--- Iterative Process ---  
Step 1: 1  
Step 2: 2  
Step 3: 6  
Step 4: 24  
Step 5: 120  
  
Factorial of 5 = 120
```



TIME COMPLEXITY :

Time complexity :-

```
int factorial(int n)
{
    int fact = 1;
    for (int i = 1; i <= n; i++)
        fact = fact * i;
    return fact;
}
```

The for loop runs n times

$i++ \rightarrow$ constant time

comparison $\rightarrow$ constant time

$fact = fact * i \rightarrow$ constant time

So, each iteration takes constant

time c .

$T(n) = c + c + c + \dots + c$

for n iterations

$T(n) = cn$

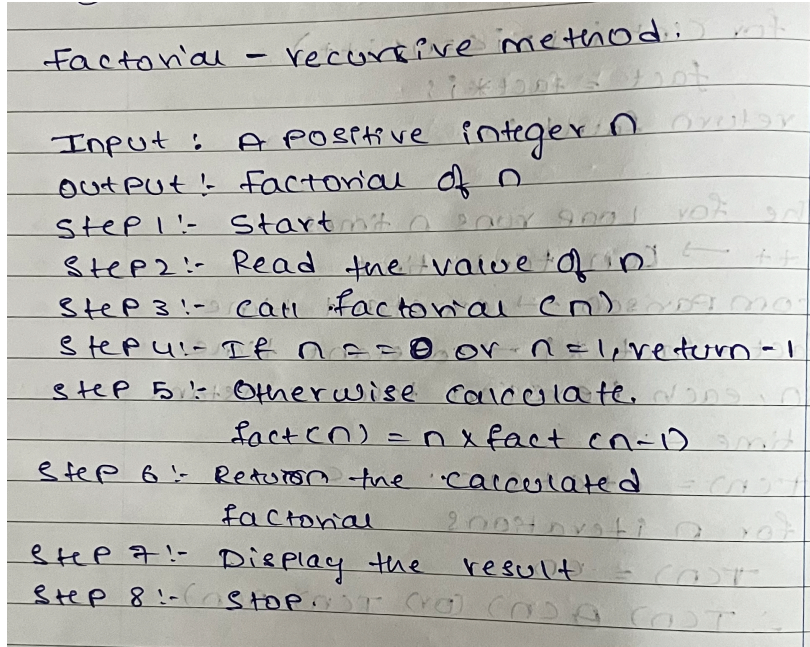
$\therefore T(n) = O(n)$ (or) $T(n) = \Theta(n)$

Best, Average, worst = $O(n)$

$T(n) = c + c + c + \dots + c$

$T(n) = cn$

$T(n) = \Theta(n)$.

ALGORITHM : // Factorial Recursive method

Factorial - recursive method:

Input : A positive integer n

Output : Factorial of n

Step 1 :- Start

Step 2 :- Read the value of n

Step 3 :- call factorial(n)

Step 4 :- If $n == 0$ or $n = 1$, return - 1

Step 5 :- Otherwise calculate,
$$\text{fact}(n) = n \times \text{fact}(n-1)$$

Step 6 :- Return the calculated factorial

Step 7 :- Display the result

Step 8 :- Stop

PROGRAM :

```
#include <iostream>
using namespace std;

long long factorial(int n) {
    // Base case
    if (n == 0 || n == 1) {
        cout << "factorial(" << n << ") = 1" << endl;
        return 1;
    }

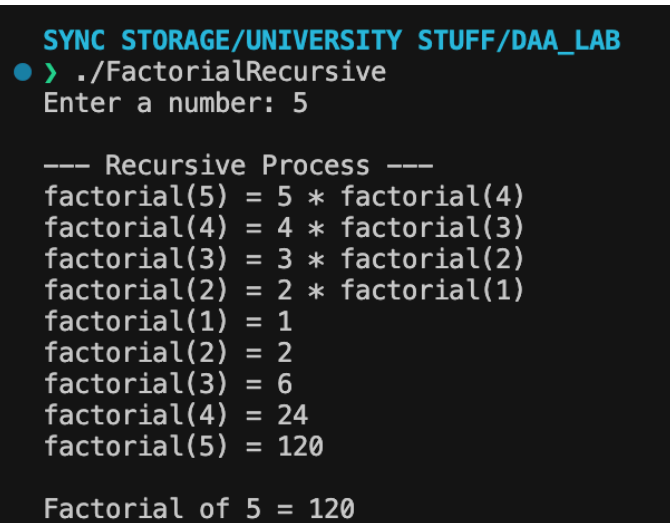
    // Show the recursive call
    cout << "factorial(" << n << ") = "
        << n << " * factorial(" << n - 1 << ")" << endl;

    // Recursive call
    long long result = n * factorial(n - 1);

    // Show the result after returning
    cout << "factorial(" << n << ") = "
        << result << endl;
    return result;
}
```

```
}  
int main() {  
    int n;  
  
    cout << "Enter a number: ";  
    cin >> n;  
  
    cout << "\n--- Recursive Process ---" << endl;  
  
    long long result = factorial(n);  
  
    cout << "\nFactorial of " << n << " = " << result << endl;  
  
    return 0;  
}
```

OUTPUT :



```
SYNC STORAGE/UNIVERSITY STUFF/DAA_LAB  
● > ./FactorialRecursive  
Enter a number: 5  
  
--- Recursive Process ---  
factorial(5) = 5 * factorial(4)  
factorial(4) = 4 * factorial(3)  
factorial(3) = 3 * factorial(2)  
factorial(2) = 2 * factorial(1)  
factorial(1) = 1  
factorial(2) = 2  
factorial(3) = 6  
factorial(4) = 24  
factorial(5) = 120  
  
Factorial of 5 = 120
```



TIME COMPLEXITY :

Time complexity :-

factorial (n)

```

1
if (n == 0 || n == 1) {
    return 1;
} else {
    return n * factorial(n-1);
}

```

$T(n) = T(n-1) + O(1)$
 $T(n) = T(n-1) + 1$
 $\therefore T(n) = T(n-2) + 2$
 $T(n) = T(n-k) + k$
 $n - k = 1$
 $k = n - 1$
 so
 $T(n) = T(1) + (n-1)$
 Since $T(1) = O(1)$
 $T(n) = O(1) + (n-1)$
 $T(n) = O(n)$
 $T_c = O(n)$

Best case :-
 $T(n) = O(1)$

Average case :-
 $T(n) = T(n-1) + c$
 $T(n) = T(n-2) + 2c$
 $T(n) = T(n-3) + 3c$
 $T(n) = T(n-k) + kc$
 $n - k = 1$
 $k = n - 1$
 $T(n) = T(1) + (n-1)c$
 $T(n) = O(n)$

Worst case :- $T(n) = O(n)$

CONCLUSION : In this practical, I implemented the factorial program using both iterative and recursive methods. I observed that both methods produce the same result, but they work differently. The iterative method uses a loop and requires less memory, while the recursive method uses function calls and is easier to understand for problems involving recursion. I also analysed their time complexity and found that both methods have $O(n)$ time complexity. Through this practical, I understood the difference between iterative and recursive approaches and how their performance and memory usage can vary.